6.8610 Spring 2026 Lecture 4 Notes: Transformers

Junru Ren with help from Codex

February 12, 2026

1 Transformers

1.1 Motivation

The lecture introduces the **Transformer** as the architecture that turned attention from a helpful seq2seq add-on into the main computational primitive. The goal is to produce strong contextualized representations for every token while avoiding the sequential bottlenecks of recurrent models.

- In Lecture 3, attention improved seq2seq models by letting a decoder look back at all encoder states instead of compressing the source sentence into one vector.
- The next step is to ask whether attention itself can do most of the modeling work.
- We want each token to build a rich context-dependent representation by deciding how much to use information from every other token in the same sequence.
- We also want a model that is highly parallelizable, unlike recurrent neural networks (RNNs) whose hidden states must be computed sequentially.
- This leads to self-attention, Transformer encoders, Transformer decoders, and practical pre-trained systems such as Bidirectional Encoder Representations from Transformers (BERT) and Generative Pre-trained Transformer (GPT) variants such as GPT-2.

1.2 Key Concepts

Definition: Self-Attention

Self-attention is a mechanism in which each token in a sequence computes how much it should attend to every token in that same sequence, including itself, and uses those weights to form a new representation.

- **Contextualized embeddings:** the main goal is not merely to score the next word, but to produce a strong representation of each token after incorporating its context.
- **Query, key, and value vectors:** each token embedding is projected into three learned views.
  - The **query** acts like a probe: what information is this token looking for?
  - The **key** describes what a token offers for matching.
  - The **value** contains the information that is actually passed forward once attention weights are decided.
- **Scaled dot-product attention:** token-to-token similarity is computed by a dot product between a query and keys, scaled before a softmax.
- **Three ways to attend:**

  - **Self-attention:** operate on one sequence and decide how much each token should attend to the other tokens in that same sequence.
  - **Masked self-attention:** still operate on one sequence, but forbid attention to future positions so the model remains autoregressive.
  - **Cross-attention:** use one sequence to attend to another, as in an encoder-decoder model.
- **Residual connections and layer normalization:** these stabilize deep networks and help preserve useful signal through many layers.
- **Positional encodings:** plain self-attention is permutation-invariant, so token order must be injected explicitly.
- **Multi-head attention:** multiple independent attention heads let the model capture different relationships in parallel.
- **Transformer encoder:** a block consisting of multi-head self-attention, a feed-forward network, residual connections, and layer normalization.
- **Transformer decoder:** similar to an encoder, but it uses masked self-attention and, in encoder-decoder settings, an additional cross-attention sublayer.
- **Types of learning:**
  - **Supervised learning:** train on labeled inputs and outputs for a specific task.
  - **Unsupervised / self-supervised learning:** train on unlabeled text by deriving targets from the data itself, such as predicting masked or future tokens.
  - **Semi-supervised learning:** combine a small amount of labeled data with a larger amount of unlabeled data.
  - **Multi-task learning:** train a shared model on multiple tasks.
  - **Transfer learning:** learn useful capabilities on one task or domain and reuse them on another.
  - **Pretraining:** the first stage of transfer learning, usually on a broad unlabeled corpus before downstream adaptation.

1.3 Core Models and Equations

Per-token projections. For input embedding x_i,

$$q_i = W_Q x_i, \qquad k_i = W_K x_i, \qquad v_i = W_V x_i.$$

The same three weight matrices are used for every token in the sequence.

Attention scores. For token i attending to token j,

$$s_{ij} = q_i^\top k_j.$$

Scaled attention weights. Let d_k be the key dimension. Then

$$a_{ij} = \mathrm{softmax}_j\left(\frac{q_i^\top k_j}{\sqrt{d_k}}\right) = \frac{\exp\left(q_i^\top k_j/\sqrt{d_k}\right)}{\sum_{\ell=1}^{n}\exp\left(q_i^\top k_\ell/\sqrt{d_k}\right)}.$$

The lecture emphasized that dividing by $\sqrt{d_k}$ keeps logits from becoming too large, which would otherwise make the softmax nearly one-hot and gradients too small.

Output representation.

$$z_i = \sum_{j=1}^{n} a_{ij} v_j.$$

For the slide example with the token "The," the lecture walked through

$$z_1 = 0.87\, v_1 + 0.12\, v_2 + 0.01\, v_3 + 0\, v_4.$$

Matrix form of self-attention. If

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

then the entire attention head can be written as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Multi-head attention (MHA). For head h,

$$\text{head}_h = \text{Attention}(QW_Q^{(h)}, KW_K^{(h)}, VW_V^{(h)}).$$

The heads are concatenated and linearly combined:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H) W_O.$$

Feed-forward neural network (FFNN) sublayer. After attention, each token representation is passed through a position-wise feed-forward network:

$$\text{FFNN}(z_i) = W_2\, \sigma(W_1 z_i + b_1) + b_2.$$

Residual connection.

$$f_{\text{residual}}(x) = f(x) + x.$$

In a Transformer block, residual connections are applied around both the attention sublayer and the feed-forward sublayer.

Layer normalization. For token representation $z_t = [z_{t,1}, \ldots, z_{t,d}]$,

$$\mu_t = \text{mean}(z_t), \qquad \sigma_t = \text{std}(z_t),$$

$$\hat{z}_{t,j} = \frac{z_{t,j} - \mu_t}{\sigma_t + \epsilon}, \qquad y_{t,j} = \gamma_j \hat{z}_{t,j} + \beta_j.$$

The lecture noted both pre-norm and post-norm variants:

$$r_t^{\text{pre}} = f(\text{LayerNorm}(z_t)) + z_t, \qquad r_t^{\text{post}} = \text{LayerNorm}(f(z_t) + z_t).$$

Positional encodings (PE). The original Transformer adds a positional vector p_i to each token embedding:

$$x_i' = x_i + p_i.$$

For sinusoidal positional encodings,

$$\text{PE}_{(\text{pos},\, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right),$$

$$\mathrm{PE}_{(\mathrm{pos},\,2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right).$$

Masked self-attention. In a decoder, future positions are masked:

$$\mathrm{MaskedAttention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V,$$

where

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i, \\ -\infty & \text{if } j > i. \end{cases}$$

This preserves autoregressive generation.

Cross-attention. In an encoder-decoder Transformer, decoder states query the encoder outputs:

$$Q^{dec} = H^{dec}W_Q, \qquad K^{enc} = H^{enc}W_K, \qquad V^{enc} = H^{enc}W_V,$$

$$\mathrm{CrossAttention}(Q^{dec}, K^{enc}, V^{enc}) = \mathrm{softmax}\left(\frac{Q^{dec}(K^{enc})^\top}{\sqrt{d_k}}\right)V^{enc}.$$

The lecture explicitly distinguished this from decoder self-attention: in cross-attention, the queries come from the current decoder layer, while the keys and values come from the encoder outputs.

BERT objectives.

- **Masked language modeling:**

$$\max_\theta \sum_{t\in\mathcal{M}} \log p_\theta(x_t \mid x_{\setminus\mathcal{M}})$$

  where $\mathcal{M}$ is the set of selected masked positions.
- **Next sentence prediction:**

$$p_\theta(\texttt{isNext} \mid s_1, s_2).$$

1.4 Algorithms / Architectures

1. Self-attention for one head

- Start with token embeddings $x_1, \ldots, x_n$.
- Project each token into a query, key, and value vector using shared matrices W_Q, W_K, W_V.
- For a fixed token i, compute dot-product scores $q_i^\top k_j$ against every token j.
- Scale the scores by $\sqrt{d_k}$ and apply a softmax to obtain attention weights a_{ij}.
- Form the new token representation $z_i = \sum_j a_{ij} v_j$.
- Repeat for all tokens; these computations can be parallelized across positions.

2. Transformer encoder block

- Add positional information to the input embeddings.
- Run multi-head self-attention to produce context-aware token representations.
- Apply a residual connection and layer normalization.
- Pass each token independently through a feed-forward network.
- Apply another residual connection and layer normalization.
- Stack several such encoder blocks to obtain stronger contextualized representations.

3. Multi-head attention

- Duplicate the attention mechanism with multiple independent parameter sets.
- Let each head compute its own attention pattern and output representation.
- Concatenate all head outputs.
- Project the concatenated representation back to the model dimension.

4. BERT pretraining

- Use only Transformer encoders.
- Treat the setup as self-supervised pretraining: the corpus is unlabeled, and the prediction targets are created from the text itself.
- Select 15% of tokens for the masked language-model objective.
- Of those selected tokens, replace 80% with `[MASK]`, 10% with a random token, and leave 10% unchanged.
- Train the model to predict the original selected tokens.
- In the original lecture material, also train next sentence prediction by deciding whether sentence 2 truly follows sentence 1.

5. BERT fine-tuning

- Keep the pretrained encoder stack.
- Treat this as supervised transfer learning: start from pretrained weights, then adapt on a smaller labeled task dataset.
- Replace the top prediction layer with a task-specific head.
- Feed downstream inputs using the same tokenization as in pretraining.
- Often freeze many BERT weights initially, then gradually unfreeze or fine-tune more of them.

6. Transformer decoder

- Use masked self-attention so token i can only attend to positions $\leq i$.
- In encoder-decoder settings, add a cross-attention sublayer where decoder queries attend to encoder outputs.
- During training, use teacher forcing: provide gold previous target tokens one step at a time.
- During generation, condition only on the prefix generated so far.

7. GPT-style decoding

- Use decoder-only Transformer blocks.
- Apply masked self-attention at every layer.
- Predict the next token autoregressively.
- Reuse the same token embedding and positional-encoding machinery across the stacked decoder blocks.

1.5 Important Intuitions from the Professor

- **Queries, keys, and values play different roles.** The lecture stressed that values are not redundant: separating them gives the model more freedom, because the same representation does not have to serve both as a matching signal and as the content being passed forward.
- **The square-root scaling is about optimization, not just notation.** Without scaling, dot products grow with dimensionality, producing overly sharp softmax distributions and poor gradient behavior.

- **Self-attention is not a brand-new concept relative to Lecture 3.** It is closely related to seq2seq attention; the main difference is that the query, key, and value all now come from the same sequence.
- **Residual connections are a safety mechanism.** They preserve the original signal so a sublayer does not need to learn a perfect transformation immediately in order for information to survive.
- **Layer normalization stabilizes both activations and training.** The professor emphasized that raw values can become too large even before thinking about backpropagation, and that this instability makes learning sensitive to initialization.
- **Parallelizability is one of the main practical reasons Transformers were so successful.** Most of the work before the softmax can be done for all tokens at once, unlike RNNs.
- **Positional encodings should be understood by their design goals more than by memorizing one formula.** The important desiderata are order information, relative-position cues, and the ability to generalize beyond lengths seen during training.
- **Multi-head attention gives the model multiple chances to discover structure.** Different heads can specialize in different patterns, such as local syntax, long-range dependencies, or punctuation-sensitive behavior.
- **BERT is primarily for representation learning, not free-form generation.** The lecture repeatedly contrasted encoder-only models, which excel at contextualized embeddings and classification, with decoder-only models, which are used to generate text.
- **What Lecture 3 called "attention" is specifically cross-attention in Transformer language.** The professor stressed that once both self-attention and cross-attention exist, the generic word "attention" becomes ambiguous.
- **Labels are scarce, so pretraining is the practical answer.** The motivation for BERT fine-tuning was not just elegance; it was that self-supervised pretraining on unlabeled text makes later supervised learning feasible when task labels are limited.

1.6 Examples

- **Cartoon self-attention example:** for the sequence "The brown dog ran," the lecture manually computed attention scores for "The" and then formed

$$z_1 = 0.87\, v_1 + 0.12\, v_2 + 0.01\, v_3 + 0\, v_4.$$

  This illustrates that a token may attend mostly to itself, somewhat to a nearby modifier, and hardly at all to unrelated tokens.
- **Permutation-invariance warning:** without positional encodings, self-attention behaves too much like a bag of words. Reordering tokens does not by itself encode the correct sequence order.
- **Machine translation:** the lecture used English-to-Czech translation to motivate Transformer decoders, masked self-attention, and cross-attention.
- **BERT masked-LM training:** a sentence such as "The brown dog ran" can be turned into "The `[MASK]` dog ran," and the encoder must recover the original token from context.
- **Sentence-pair classification with BERT:** the classification token `[CLS]` can be used as a summary representation for tasks with one label per input sequence or per sentence pair.

- **Token-level prediction with BERT:** downstream tasks such as tagging can use the contextualized representation at each output position rather than a single global representation.
- **Causal masking in GPT-style models:** when generating token t, the model is allowed to look at tokens $1, \ldots, t-1$ but not at future tokens $t+1, \ldots, n$.
- **Three ways to attend:** self-attention operates within one sequence, masked self-attention does the same with future positions blocked, and cross-attention lets a decoder attend to a separate encoder sequence.

1.7 Common Pitfalls / Limitations

- **No positionality by default:** raw self-attention is permutation-invariant and must be augmented with positional information.
- **Quadratic cost in sequence length:** standard self-attention requires interactions between every pair of positions, which scales as $O(n^2)$.
- **Attention alone does not provide nonlinearity.** The feed-forward network remains important for expressive power.
- **Optimization instability:** without scaling, residual connections, and normalization, deep attention networks are much harder to train.
- **Encoder-only models cannot directly generate arbitrary sequences.** BERT learns excellent contextualized embeddings but is not itself a decoder-style language generator.
- **Fine-tuning can fail if preprocessing changes.** The lecture specifically warned that downstream inputs should use the same tokenization regime as pretraining.
- **Very long contexts are expensive.** The professor noted that large context windows require additional engineering tricks beyond the vanilla attention computation.

1.8 Connections to Other Lectures

- This lecture builds directly on Lecture 3's seq2seq attention. Cross-attention in a Transformer decoder is conceptually the same operation as the encoder-decoder attention introduced there.
- It also resolves a central limitation from Lecture 3: RNNs process sequences sequentially, while self-attention gives every token direct access to every other token and is much easier to parallelize.
- Compared with Lecture 2's word2vec and bag-of-words representations, Transformers produce **contextualized** token representations rather than one static vector per word type.
- Compared with Lecture 1's general language-model framing, this lecture provides the architecture that ultimately became the dominant implementation for modern large language models.
- BERT and GPT-2 preview two important branches of later NLP systems: encoder-style pretrained models for understanding tasks and decoder-style models for autoregressive generation.

1.9 Exam-Relevant Summary

- A self-attention head computes

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i,$$

then uses scaled dot-product attention to produce z_i.

- Core attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- The $\sqrt{d_k}$ factor prevents overly large logits and overly sharp softmax distributions.
- Values are separate from queries and keys because the model benefits from different representations for matching versus content passing.
- A Transformer encoder block consists of multi-head self-attention, residual connections, layer normalization, and a feed-forward network.
- Positional encodings are necessary because self-attention alone is permutation-invariant.
- Multi-head attention lets different heads capture different relations in parallel.
- Types of learning discussed in this lecture include supervised, unsupervised/self-supervised, semi-supervised, multi-task, and transfer learning; pretraining is the first stage of transfer learning.
- Standard self-attention is $O(n^2)$ in sequence length, but it is still highly practical because the computations are parallelizable.
- BERT is an encoder-only Transformer pretrained with masked language modeling and, in the original version, next sentence prediction.
- BERT pretraining is self-supervised on unlabeled text; BERT fine-tuning is supervised transfer learning on a labeled downstream task.
- Original BERT masking rule: choose 15% of tokens; of those, 80% become `[MASK]`, 10% become random tokens, and 10% stay unchanged.
- BERT is typically fine-tuned by attaching a task-specific head while preserving the pretrained encoder.
- A Transformer decoder uses **masked self-attention**; an encoder-decoder Transformer also uses **cross-attention**, where decoder queries attend to encoder keys and values.
- GPT-2 is decoder-only: it generates text autoregressively and does not use encoder-side cross-attention.